

Secret Mysteries of Automated Execution

Alan Richardson

@eviltester

- www.EvilTester.com
- www.CompendiumDev.co.uk

Secrets & Mysteries of Automated Execution

Hopefully stuff not covered later

There are many things we don't know:

- Does Bigfoot exist?
- Are UFOs Alien Spacecraft?
- Is there a monster in Loch Ness?
- Does the application still function after this code change?

**Executing Application
functionality... we've
pretty much got a good
handle on that.**

Or do we?

**How do you know how
well your automated
execution works?**

It is not how well your automated execution works when everything is working well that tells you how well your automated execution works. It is how well your automated execution works when things are not working well that tells you how well your automated execution works.

**Your Automated
Execution is only as good
as it deals with difference**

Difference?

- changes to application
- slow / partial environments
- maintenance, refactoring
- wrong data

What is the difference that make a difference?

What is the response to the difference?

- report errors?
- self healing?
- debug information?
- take another path?
- skip step?
- silent failure?

WIIFM

- Generalisations
- Simple Automation Models
- Understand "The Why"
- Techniques Illustrated (with Code)
- Reinforcement of Important Concepts

Decisions & Disagreements

- Right / Wrong
- Appropriate / Less Appropriate
- Decided upon / Decided against

What are the Secret Differences Between Coding an Application and Automated Execution?

The Paranoia Principle



Automated Execution Would Avoid....

```
class MyApp {  
    public static void main(String[] args) {  
        try{  
            new myApplication().run(args);  
  
        }catch(Throwable t){  
            // never let the user know  
            // there was a problem  
        }  
    }  
}
```

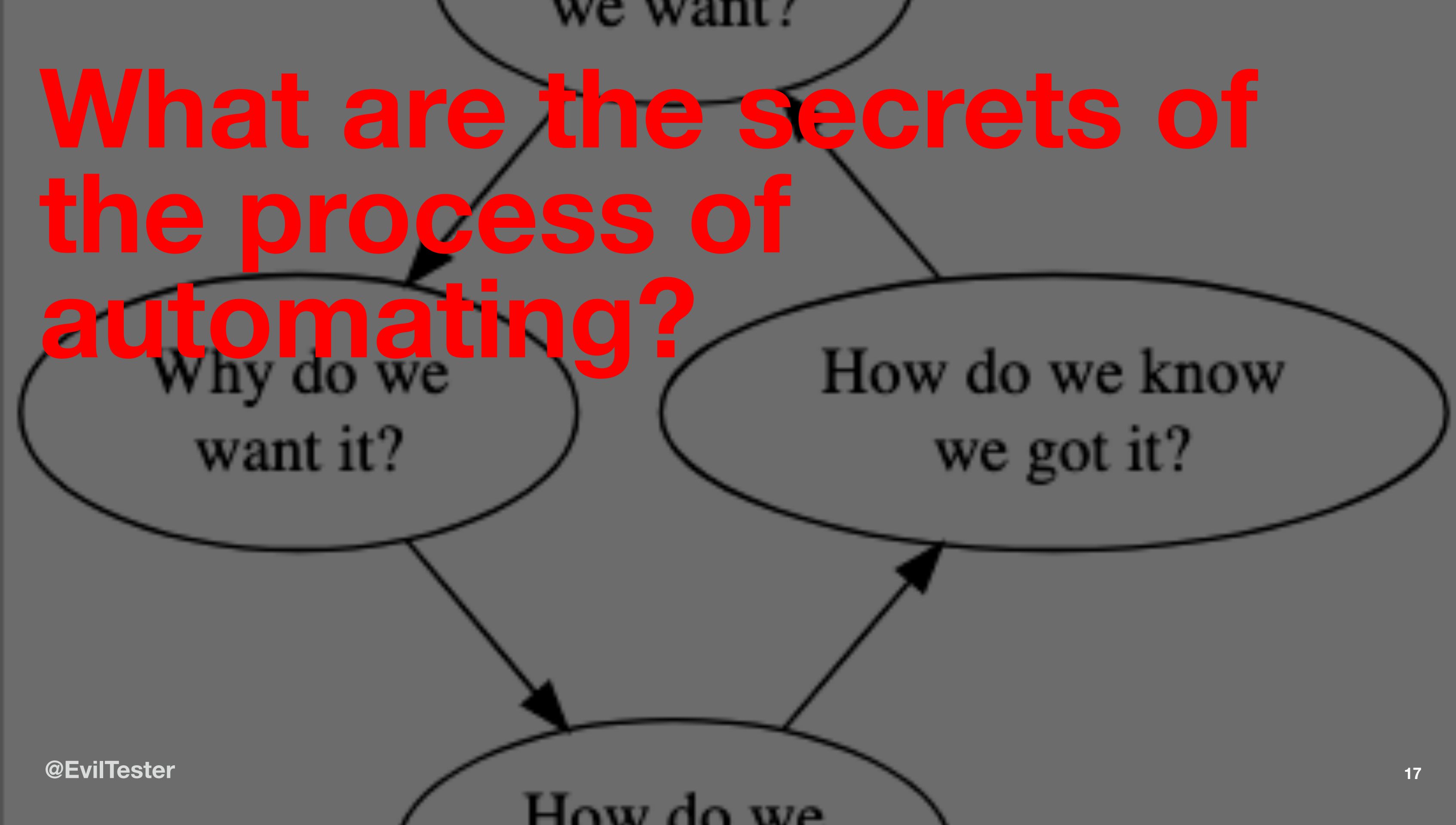
Code Automated Execution for Paranoia

- **THEY** are lying to you
 - Check **ALL** assumptions/
preconditions
- **THEY** want to trap you
 - Don't move unless sure
- **THEY** will twist your words
 - Log your actions! Take
screenshots!
- **THEY** falsify evidence
 - check and log all post
conditions before moving
- **THEY** want a scapegoat
 - throw exceptions, fail the
run, avoid blame

@EvilTester

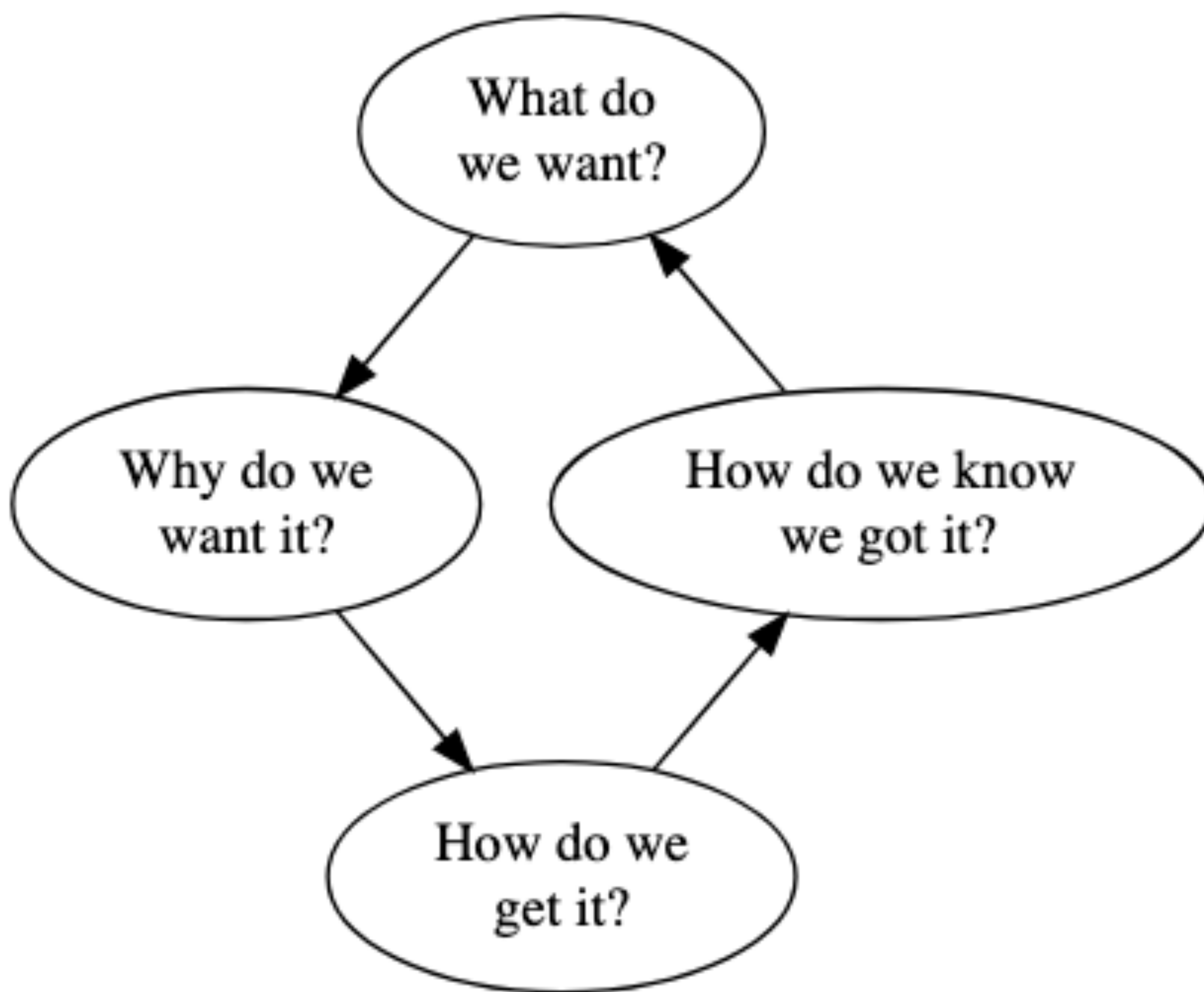


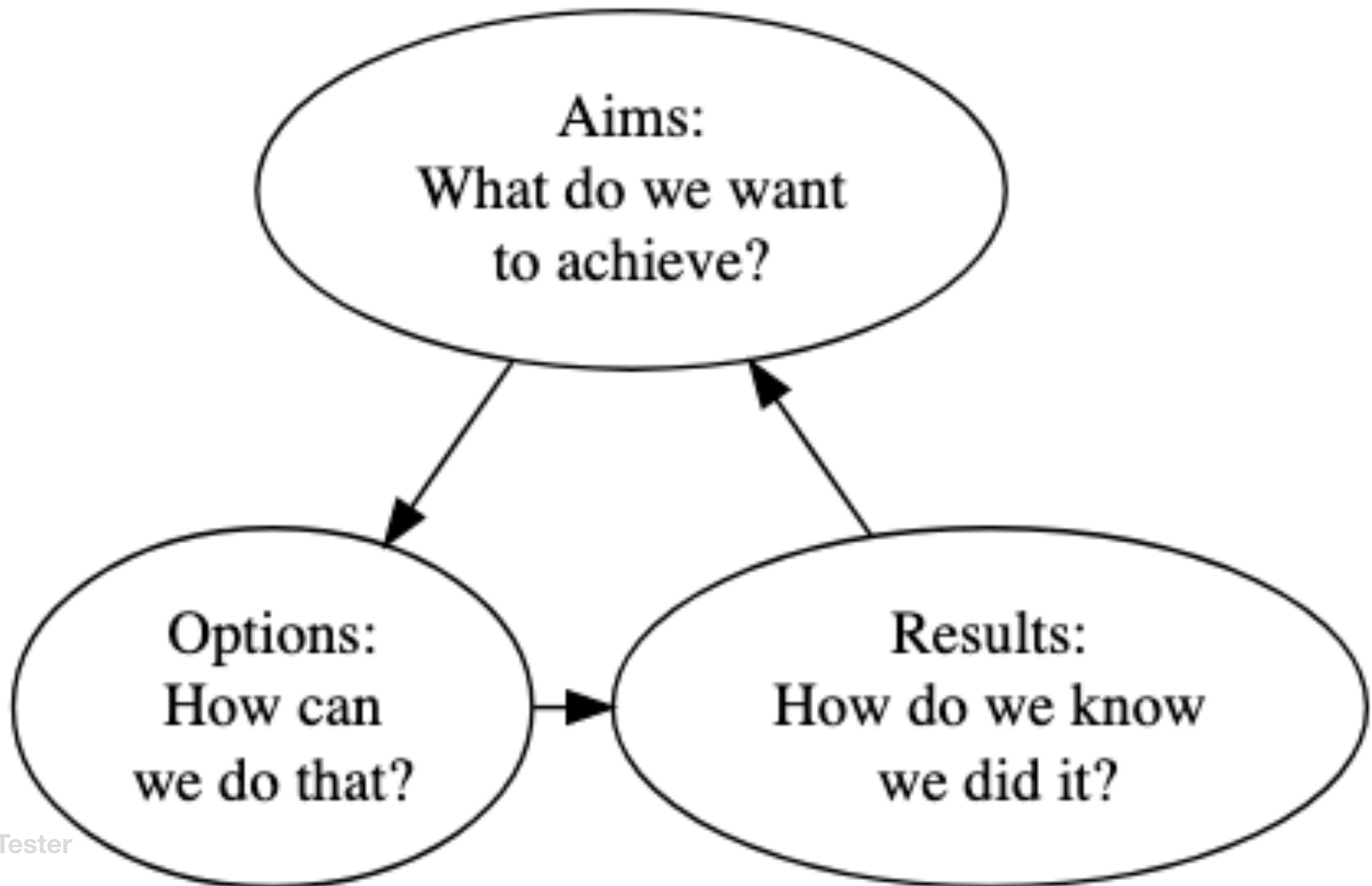
What are the secrets of the process of automating?



Processes & Tools

- Requirements and Specifications
- Programming Languages, Compilers
- Libraries
- Patterns, SOLID
- TDD, BDD, Unit Testing
- Exploratory Testing
- Automated Execution
- etc.





Why Say Automated Execution?

Why Say Automated Execution?

Automated Execution covers:

- build process, release process, CI/CD
- Development tools, IDEs, Static Analysis
- Environment Setup
- Monitoring, Logging, Alerting
- Exploratory Testing Support
- Test Automation
- etc.

Secret of Automated Execution

Automating is a means to an end,
not the end itself.

Automated Execution Build Process

The aim is not to automate the build process.

The aim is to:

- make the build process faster, repeatable, objective, transparent.**
- allow humans to stay in flow,**
- etc.**

Automated Execution For Testing

The aim is not to "automate all testing".

The aim is to:

- automate assertions which we want to regularly check that they still apply
- cover regular paths with lots of data
- etc.

Secret : Bake Automate Execution In

Accept that automating is part of our work because it is what humans do.

**What are the hidden
secret fundamental
concepts of automating?**

Automating Concepts

- Automated Execution
 - making something do something without human intervention
- Assertion
 - detecting if what should happen, has happened
 - *more rare* detecting if something that should not have happened has happened.

What do we automate?

- systems?
- applications?
- code?
- interfaces?



Automate What? Stuff.

- **todomvc.com**
- **browser**
- **javascript**
- **elements**
- **mouse clicks**
- **keyboard**



We automate using models and abstractions

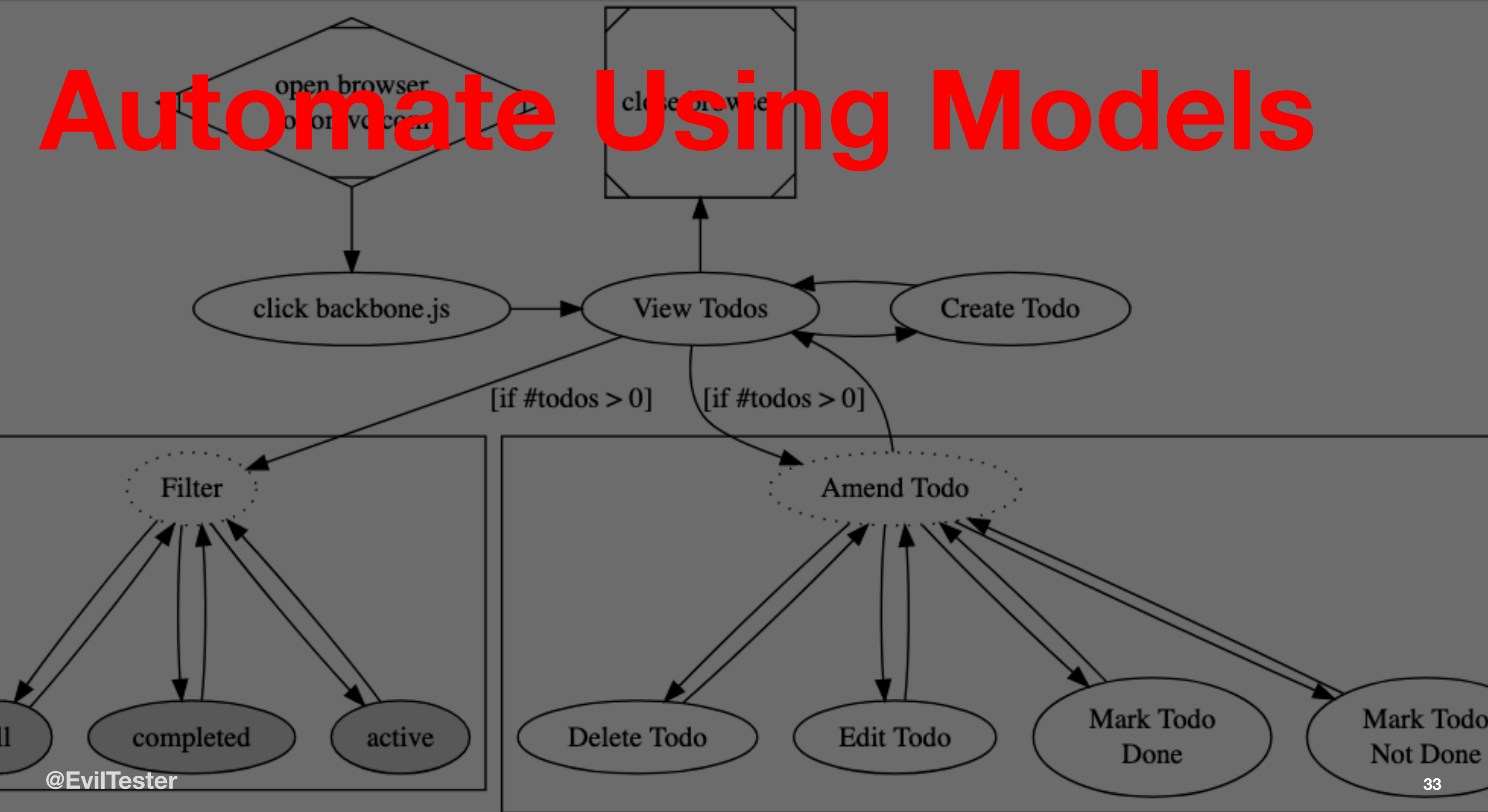
- representations of systems**
- representations of applications**
- abstractions of browsers**
- abstractions of interactions**

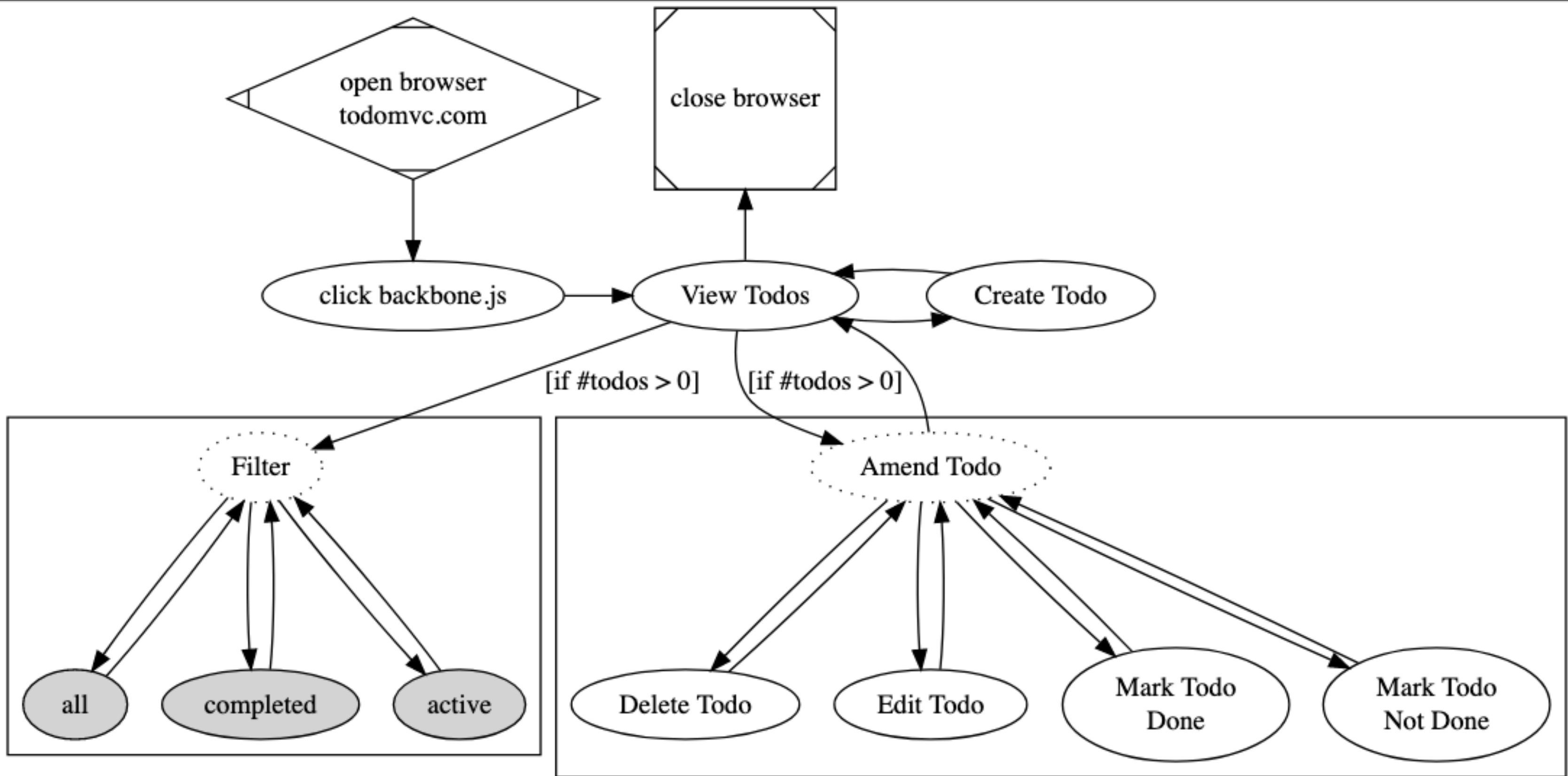
Are you aware of the risks?

because you are automating?

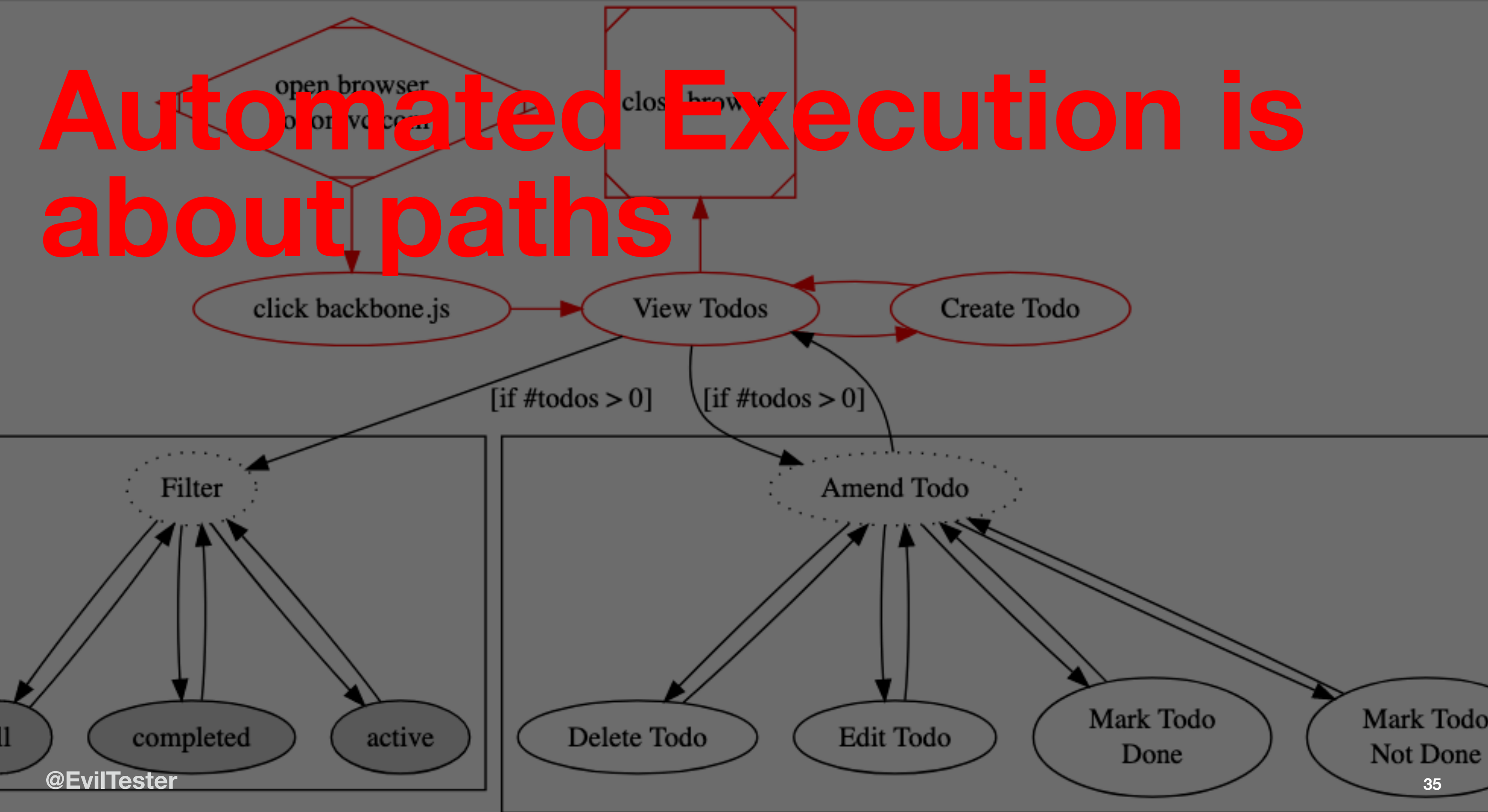
in how you are automating?

Automate Using Models



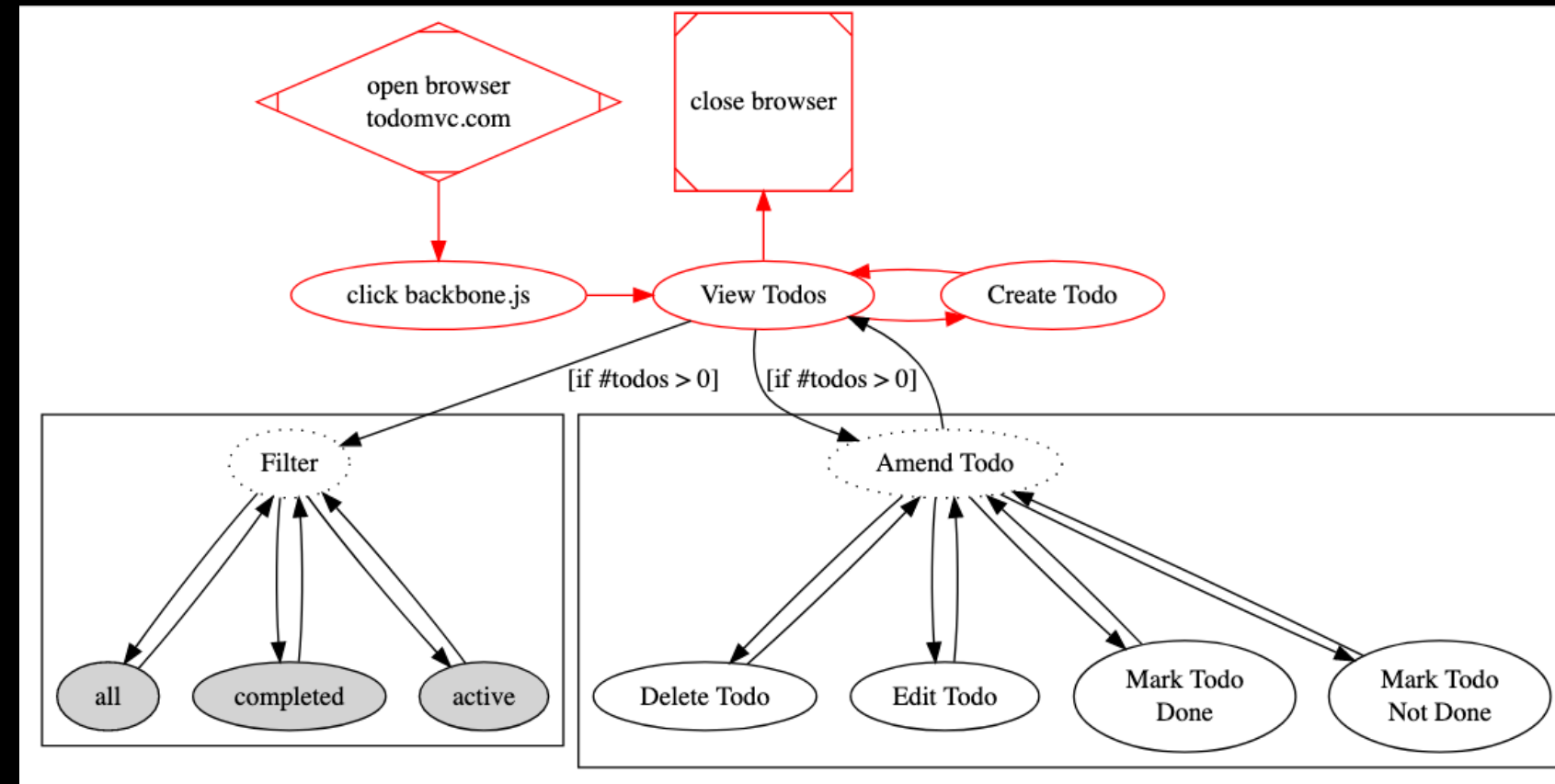


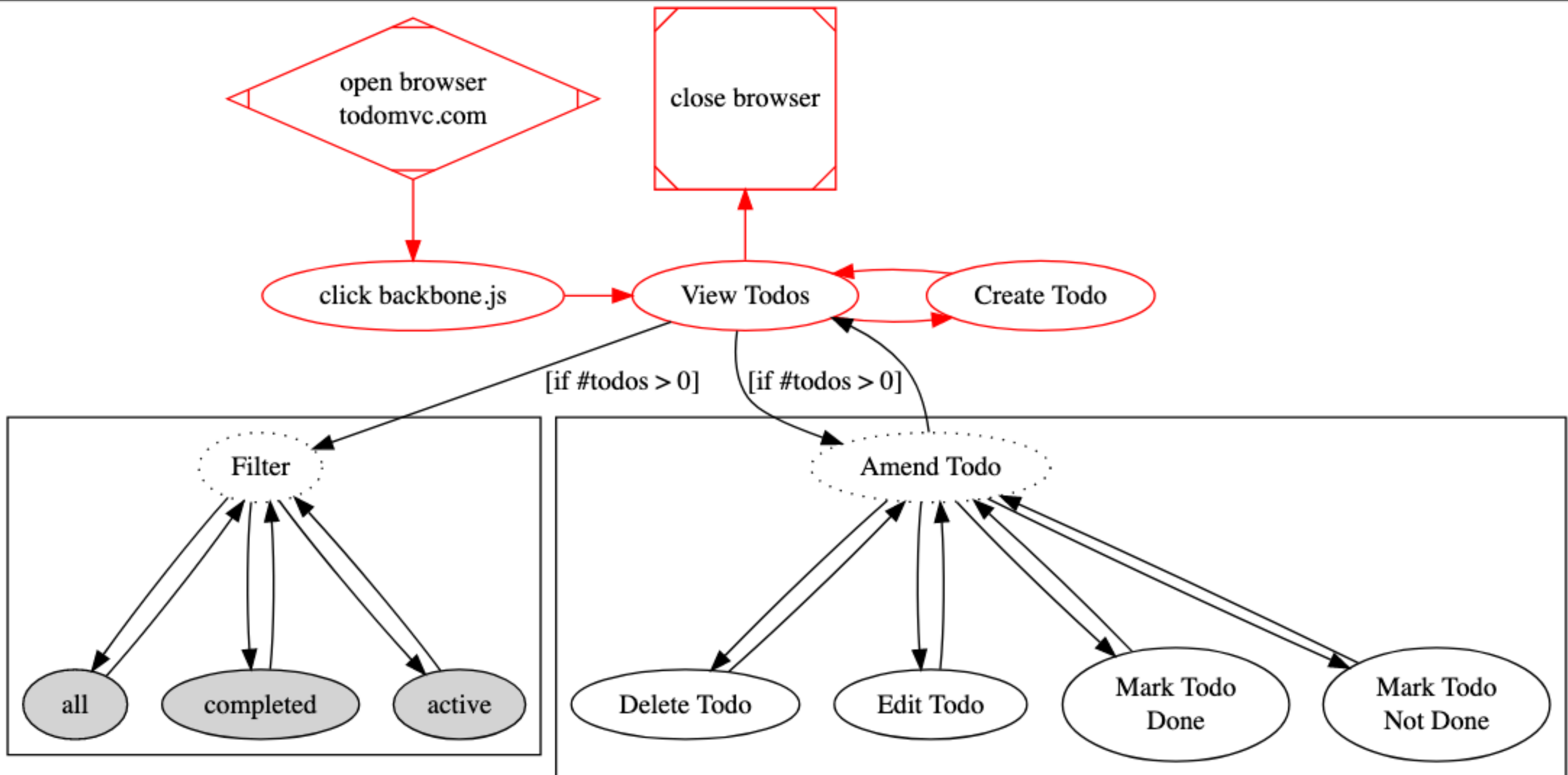
Automated Execution is about paths



Automated Execution is about paths

An executable traversal of a defined path through a system with data.





How do we do it?

Tools & Code

Path and Data Combined

```
WebDriver driver = new ChromeDriver();
driver.get(
    "http://todomvc.com/examples/backbone/");

WebElement createTodo =
    driver.findElement(
        By.className("new-todo"));
createTodo.click();
createTodo.sendKeys("new task");
createTodo.sendKeys(Keys.ENTER);

driver.close();
```

How do we know we did it?

Assertions

```
@Test
public void canCreateATodo(){
    WebDriver driver = new ChromeDriver();
    driver.get("http://todomvc.com/examples/backbone/");

    WebElement createTodo = driver.findElement(
        By.className("new-todo"));
    createTodo.click();
    createTodo.sendKeys("new task", Keys.ENTER);

    Assertions.assertTrue(driver.findElement(
        By.className("filters")).isDisplayed());

    int newToDoCount = driver.findElements(
        By.cssSelector("ul.todo-list li")).size();

    Assertions.assertEquals(1, newToDoCount);
    driver.close(); }
```

Concepts

- An Aim
- A Path
- Invariant Data - controls the path to be executed
- Variant Data
- Ability to automate the system
- Assertions

Whatever we are doing for automating - we are doing this.

What, How, How

- What do we want?
 - An Aim
- How do we get it?
 - A Path
 - Invariant & Variant Data
 - Ability to automate the system
- How do we know we got it?
 - Assertions

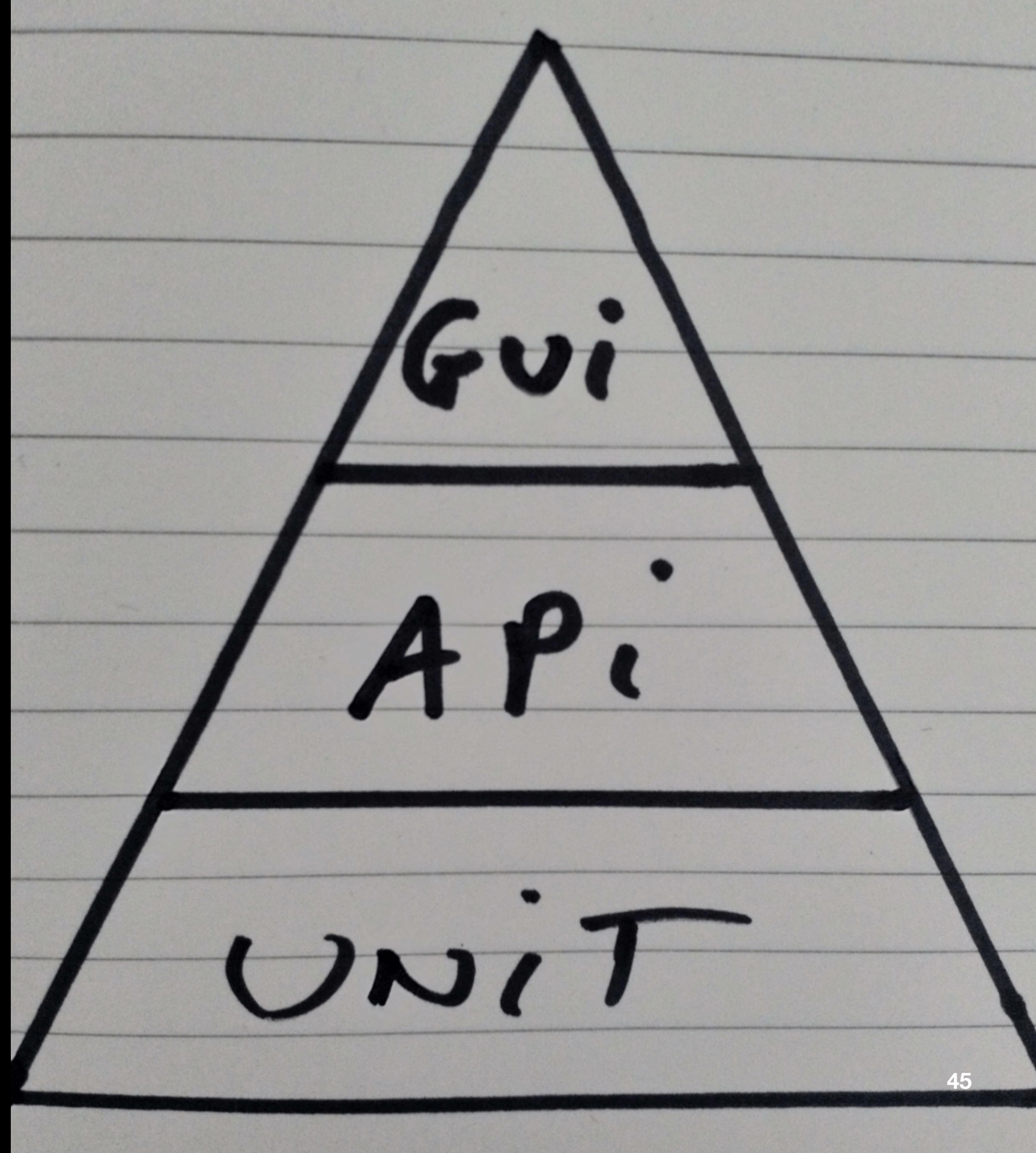
**But that example
automated the GUI and
we know that is to be
avoided**

Automation Pyramid

Mass?

Ease?

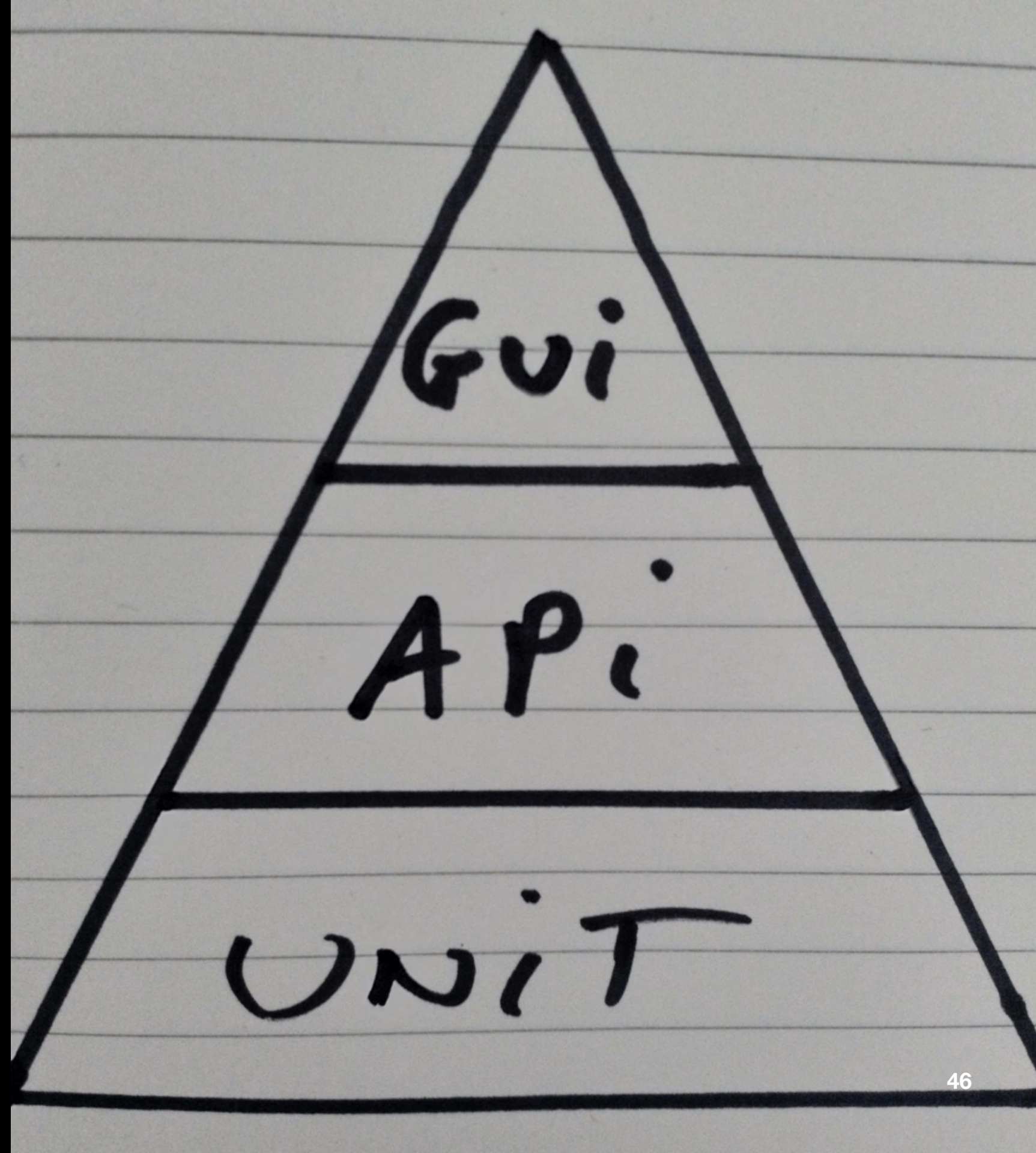
Speed?



What to Automate?

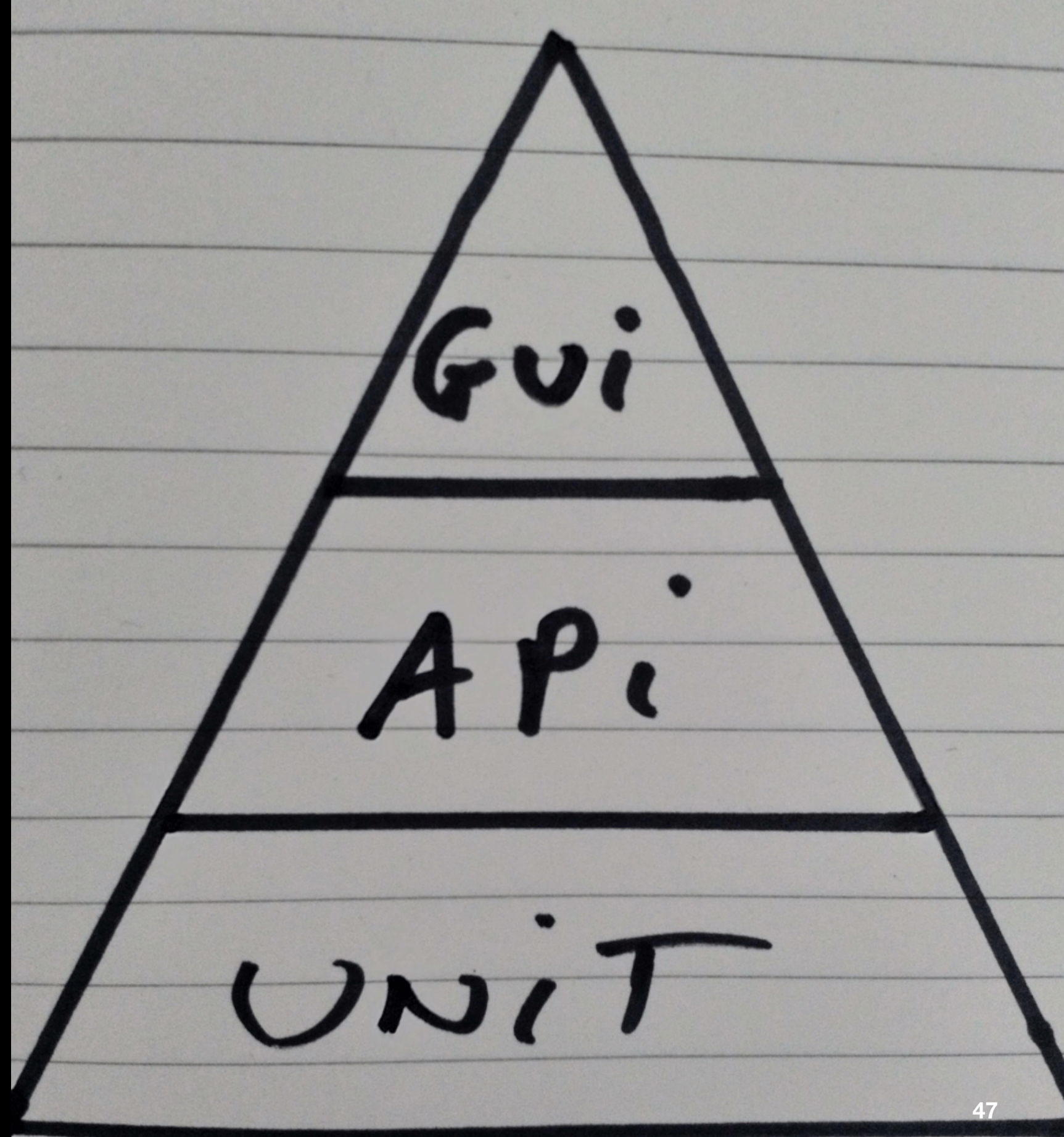
coverage not
layers

using interfaces
and covering
functionality



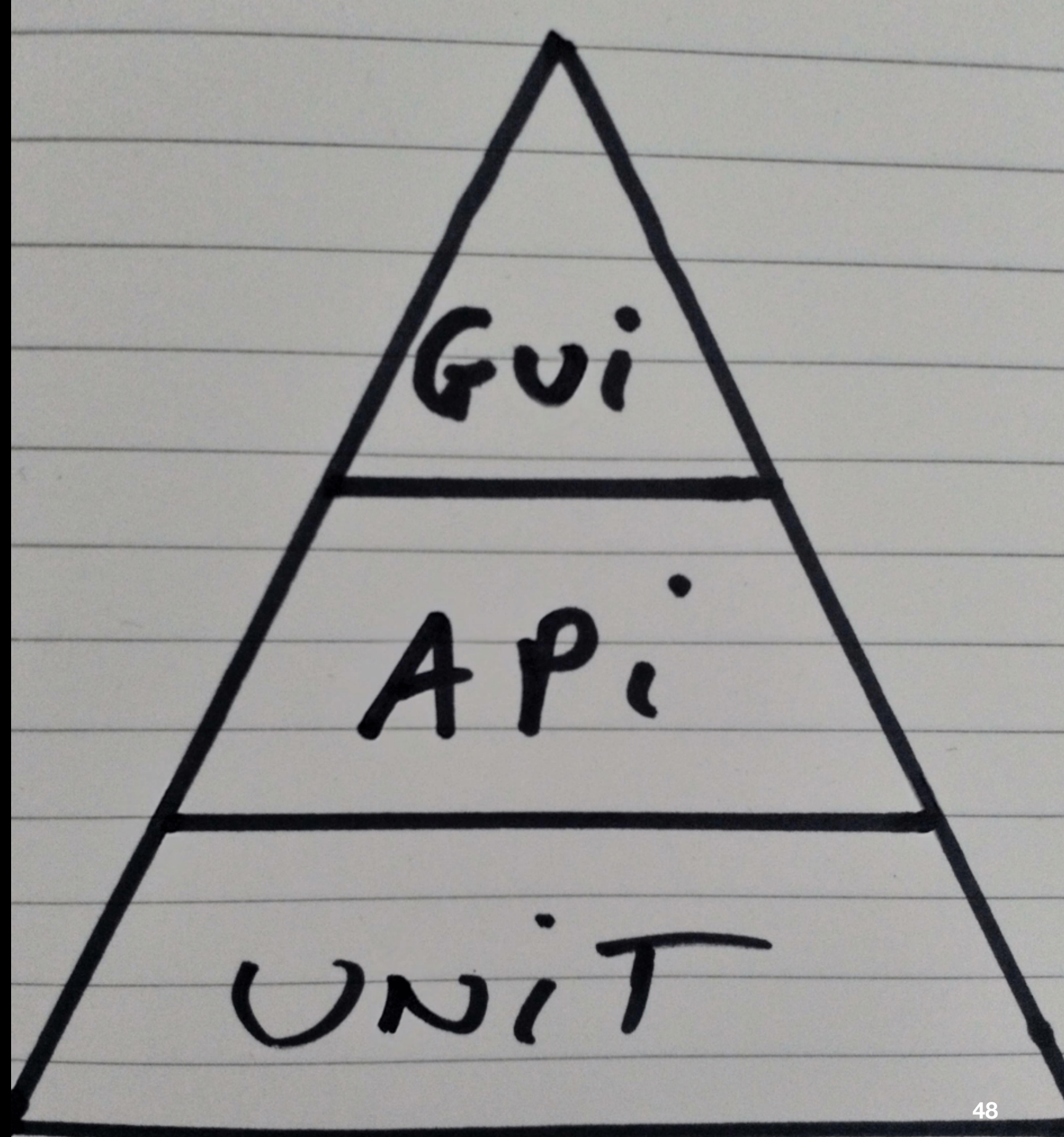
**What about
easy to
automate?**

**are some layers
and functionality
easier to
automate than
others?**



**Ease, does not
trump coverage.**

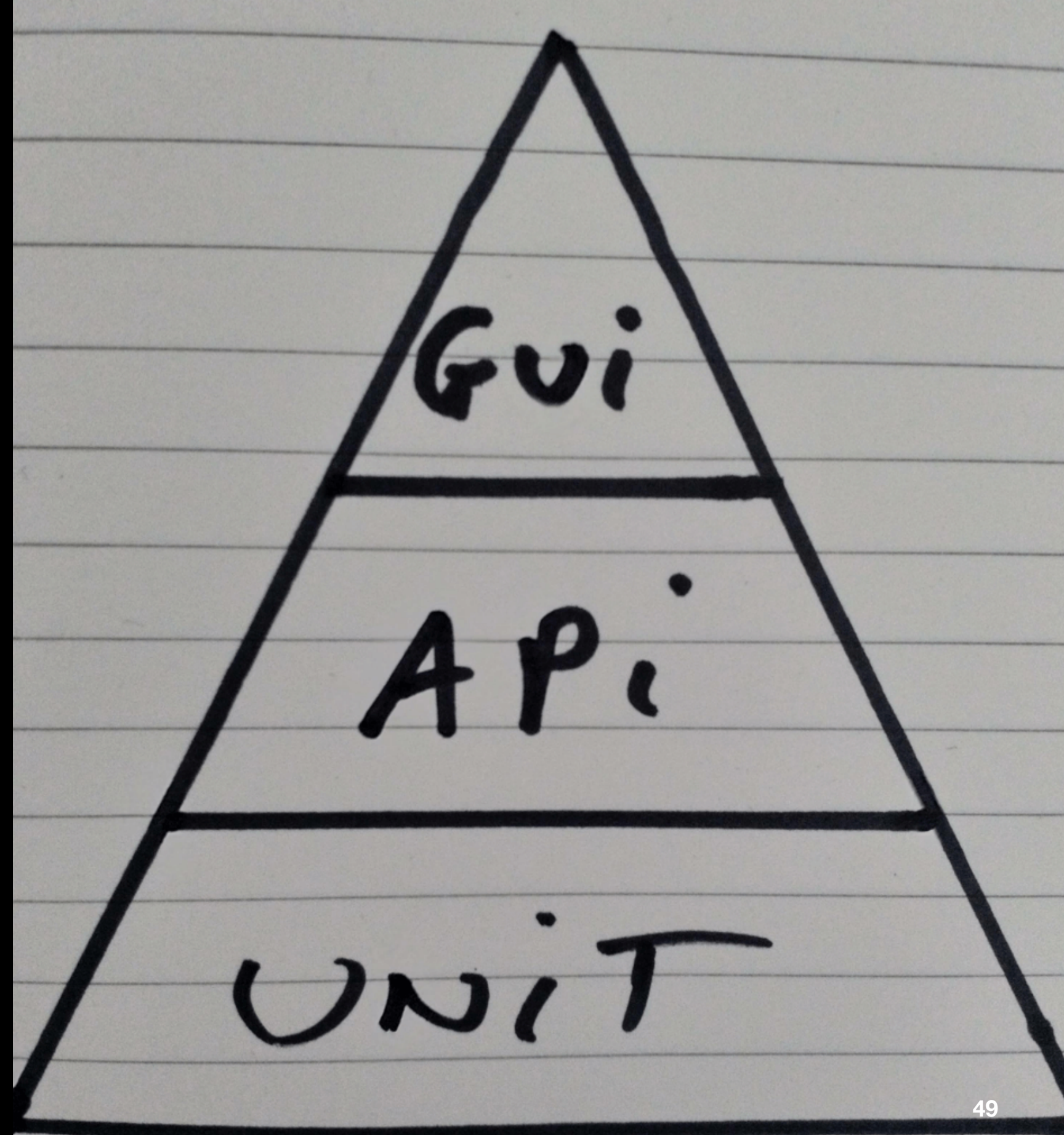
**Ease might cause
us to re-architect,
to make it
possible to
achieve coverage.**



**What do we want
to know?**

**What do we care
about?**

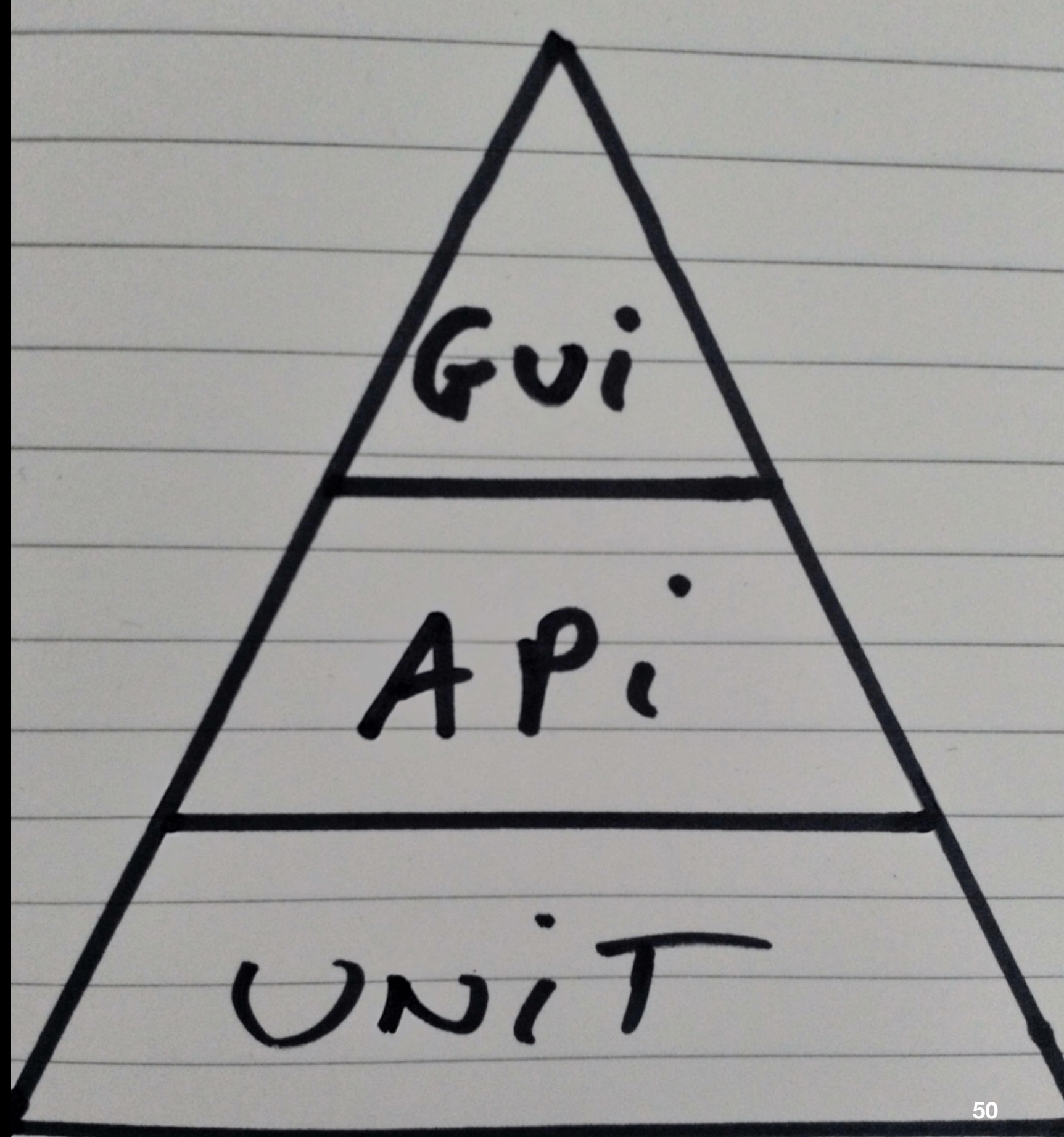
**To get the best
answers to that...**



What are the best questions to ask?

Where is the best place to ask those questions?

What is the best way to ask it?



What is the secret of making automated execution robust?

Or...

How do some teams avoid flaky automation?

How do some teams avoid intermittent build failures?

```
Thread.sleep(5000);
```

Synchronisation

Is it ready?

Is it complete?

Polling

Synchronise on State not the Passage of Time

Synchronising on state requires...

- ability to 'see' the state
 - observability, monitorability
- know what is the correct state
- act in a timely fashion while it is in that state
- timeout so we don't wait for ever

But a timeout is time

```
Thread.sleep(5000);
```

- **pause, regardless, fail on future step**

```
wait.until(componentIsReadyToUse(), 10);
```

- **poll, timeout on maximum time reached**

Wait for multiple states

```
wait.until(componentIsRendered(), 10);  
wait.until(buttonsAreVisible(3), 10);  
wait.until(buttonIsClickable(3), 10);
```

- **multiple waits in sequence, instead of time increase**
- **check all your assumptions/preconditions**

Architecture Does Not Rule Out Synchronisation

- **API**
 - **timeout on method calls**
 - **poll queues for results**
 - **poll DB for state prior to issuing another call**

@Test vs Automated Execution Does Not Rule Out Synchronisation

- **Build & Deploy Process**
 - wait for DB to be available before writing
 - have all Tables been created as expected before trying to write data
 - wait for previous step to complete, before next
 - wait for file to be readable before moving to next step

Synchronisation Failures Should Help Debug Failure

What is the secret to trusting automated execution?

**If you've only seen it pass,
make it fail**

Once it is working...

Change the system/environment:

- to check every assertion fails**
- to check wait can timeout if condition not met**
- to check every step fails if the app is different**

this is often harder than writing it in the first place.

What is the secret of using BDD for automated execution?

We don't use BDD for Automated Execution

**We can use BDD
Classified Tooling**

**Because they support Executable
DSL creation**

Multiple Representations and Multiple Tools

— If you use Cucumber, don't use it for everything

Write Automated Execution at the Correct Domain

Inappropriate Domain Level

Scenario: Should create a TODO through API

Given request body loaded from `"/resources/api/createtodo.json"`

And content type is `"application/json"`

When a POST request to `"/todos"`

Then status code is 201

And response contains `"id"` with value `"my-todo-guid"`

And response contains `"title"` with value `"created todo"`

Feature Steps delegate to other layers of abstraction

Feature: We can create and edit To Do lists in ToDoMvc

Scenario: Create a ToDo Item

Given a user opens a blank ToDoMVC page

****When the user creates a todo "new task"****

Then they see 1 todo item on the page

```
@Given("^the user creates a todo \"([^\"]*)\"$")
```

```
public void the_user_creates_a_todo(String todoName)
```

```
    throws Throwable {
```

```
    final int count = page.getTodoItems().size();
```

```
    user.createNewToDo(todoName);
```

```
    assertThat(page.getTodoItems().size(), is( count+1));}
```

“The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise.”

Edsger W. Dijkstra, The Humble Programmer

**The Secrets are things we
already know:
Prioritisation, Abstraction,
Modelling, Flexibility,
Preconditions/
PostConditions, etc.**

**That doesn't mean it's
easy.**

**But it does mean we can
do it.**

**The Secrets are things we
already know. That
doesn't mean it's easy.
But it does mean we can
do it.**

Questions?

@EvilTester

EvilTester.com

End Credits

- www.eviltester.com
- [@eviltester](https://twitter.com/eviltester)
- www.youtube.com/user/EviltesterVideos
- www.compendiumdev.co.uk
- uk.linkedin.com/in/eviltester

Follow

- **Linkedin - @eviltester**
- **Twitter - @eviltester**
- **Youtube - EvilTesterVideos**
- **Instagram - @eviltester**
- **Github - @eviltester**
- **Slideshare - @eviltester**
- **Facebook - @eviltester**
- **Pinterest - @eviltester**

BIO

Alan is a test consultant who enjoys testing at a technical level using techniques from psychotherapy and computer science. In his spare time Alan is currently programming a **multi-user text adventure game** and some **buggy JavaScript games** in the style of the Cascade Cassette 50. Alan is the author of the books "**Dear Evil Tester**", "**Java For Testers**" and "**Automating and Testing a REST API**".

Visit EvilTester.com

**Alan's main website is www.eviltester.com and he
blogs at blog.eviltester.com**